UNIVERSIDAD AUTÓNOMA DE BUCARAMANGA

FACULTAD DE INGENIERÍA

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE (SRS)

Plataforma Multimodal de Micromovilidad, Logística y Venta de Vehículos Urbanos

Versión 1.1 — Primer avance (revisión)

INGENIERÍA DE SOFTWARE II

PRESENTAN:

Jaime Alejandro Vega Barbosa – U00178766

José Alejandro Téllez Prada – U00060479

David Santiago González Nigrinis – U00179507

DOCENTE:

Daniel Arenas Seeley

PROGRAMA

Ingeniería de Sistemas

Colombia, Bucaramanga, Santander

08 – 15 de septiembre de 2026

Declaración de uso de IA generativa: *se empleó Claude (Anthropic) como apoyo para estructurar el documento, organizar los requisitos según la plantilla SRS suministrada por la asignatura y revisar la redacción técnica. Las recomendaciones de la IA fueron evaluadas críticamente por los integrantes del equipo antes de incluirlas; los conceptos de base se contrastaron con los apuntes de clase, la bibliografía sugerida por el docente, la descripción propia del modelo de negocio y el prototipo funcional (urban-mobility-hub) desarrollado por el equipo.*

1. Introducción

1.1 Propósito

Este documento especifica el comportamiento funcional y no funcional del primer avance de la Plataforma Multimodal de Micromovilidad, Logística y Venta de Vehículos Urbanos. Está dirigido al equipo de desarrollo, al docente evaluador y a cualquier interesado que requiera el detalle preciso de lo que el sistema debe hacer antes de continuar con las siguientes iteraciones de diseño e implementación.

1.2 Alcance

El presente SRS aplica al primer avance del proyecto, materializado en el prototipo funcional urban-mobility-hub (PWA construida con TanStack Start, React 19 y Tailwind CSS 4), compuesto por tres vistas: una landing page pública (/), una aplicación de mapa para el usuario final (/app) y un panel de telemetría de tipo Digital Twin para el operador (/admin). En esta etapa el sistema no opera con vehículos físicos: la flota, los hubs y la telemetría se simulan mediante gemelos digitales, actualmente representados como datos mock en memoria y, en la arquitectura objetivo, soportados sobre hardware IoT con placas Raspberry Pi. El alcance funcional incluye: catálogo y venta de vehículos, alquiler estación a estación y a domicilio, mapa interactivo multicapa con seguimiento en tiempo real, desbloqueo mediante código QR con verificación de casco por cámara (y reconocimiento facial como mecanismo alterno), y un módulo de seguridad IoT que detecta y alerta sobre caídas, pérdida de GPS, ausencia de casco y batería crítica. La planeación del proyecto se apoya en una estimación teórica mediante Puntos de Función y el modelo COCOMO, aterrizada a un equipo de trabajo real de tres (3) personas.

1.3 Definiciones, acrónimos y abreviaturas

SRS: Software Requirements Specification – Especificación de Requisitos de Software.

IoT: Internet of Things – Internet de las Cosas.

Digital Twin (gemelo digital): representación virtual de un vehículo o hub, con los mismos atributos de estado (posición, batería, ocupación, alertas) que tendría el activo físico, usada para simular su comportamiento antes de contar con la flota real.

Hub: punto físico de recolección/entrega y carga de vehículos, con capacidad total, vehículos ocupados y vehículos en carga.

Trigger de seguridad: regla que evalúa el estado de un vehículo y genera una alerta IoT cuando detecta uno de los siguientes eventos: sin casco, caída/impacto, pérdida de señal GPS o batería crítica.

DFD: Diagrama de Flujo de Datos.

COCOMO: Constructive Cost Model, modelo de estimación de esfuerzo y costo de desarrollo de software.

Puntos de Función: técnica de estimación del tamaño funcional del software a partir de sus funcionalidades.

MQTT: protocolo de mensajería ligero utilizado para la comunicación entre dispositivos IoT y el backend.

API: Application Programming Interface.

GPS: Global Positioning System.

QR: Quick Response, código utilizado para el desbloqueo de vehículos.

PWA: Progressive Web App.

1.4 Referencias

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- Plantilla SRS.dot (Rational Unified Process), suministrada por la asignatura Ingeniería de Software II.
- Repositorio del prototipo funcional: github.com/Joserraldo/urban-mobility-hub (rama principal, primer avance).
- Documento anexo: Análisis de Riesgos del Desarrollo – Plataforma Multimodal de Micromovilidad, Logística y Venta de Vehículos Urbanos.
- Documento anexo: Plan de Desarrollo de Software – Plataforma Multimodal de Micromovilidad, Logística y Venta de Vehículos Urbanos.

1.5 Resumen del documento

La sección 2 presenta la descripción general del producto, sus funciones principales, los tipos de usuario, las restricciones del proyecto y un resumen de los riesgos de mayor criticidad. La sección 3 detalla los requisitos específicos del sistema por módulo funcional, los requisitos no funcionales, los componentes de terceros, las interfaces del sistema y las condiciones legales y de licenciamiento aplicables.

2. Descripción general

2.1 Perspectiva del producto

La plataforma es un producto nuevo e independiente que integra en una sola solución la venta de vehículos urbanos (patinetas eléctricas, bicicletas eléctricas y drones), su alquiler con seguimiento en tiempo real, y un módulo de seguridad IoT durante el trayecto. En este primer avance el producto se limita al portal de negocio, la landing page y la aplicación de mapa del prototipo urban-mobility-hub; el comportamiento de la flota se simula mediante gemelos digitales que comparten una única fuente de datos entre el mapa y el panel de telemetría, anticipando la futura sustitución de esa fuente por un backend conectado a hardware real.

La Figura 1 muestra el diagrama de contexto del sistema. La plataforma se representa como un óvalo; los rectángulos son las entidades externas (usuario final, operador y vehículo IoT); y los archivos lógicos se dibujan con dos líneas horizontales, uno por cada archivo lógico interno (ILF) y externo (EIF). Cada flecha está rotulada según su tipo: entradas (EI), salidas (EO) y consultas (EQ), con los mismos nombres usados en la estimación por Puntos de Función del Plan de Desarrollo.

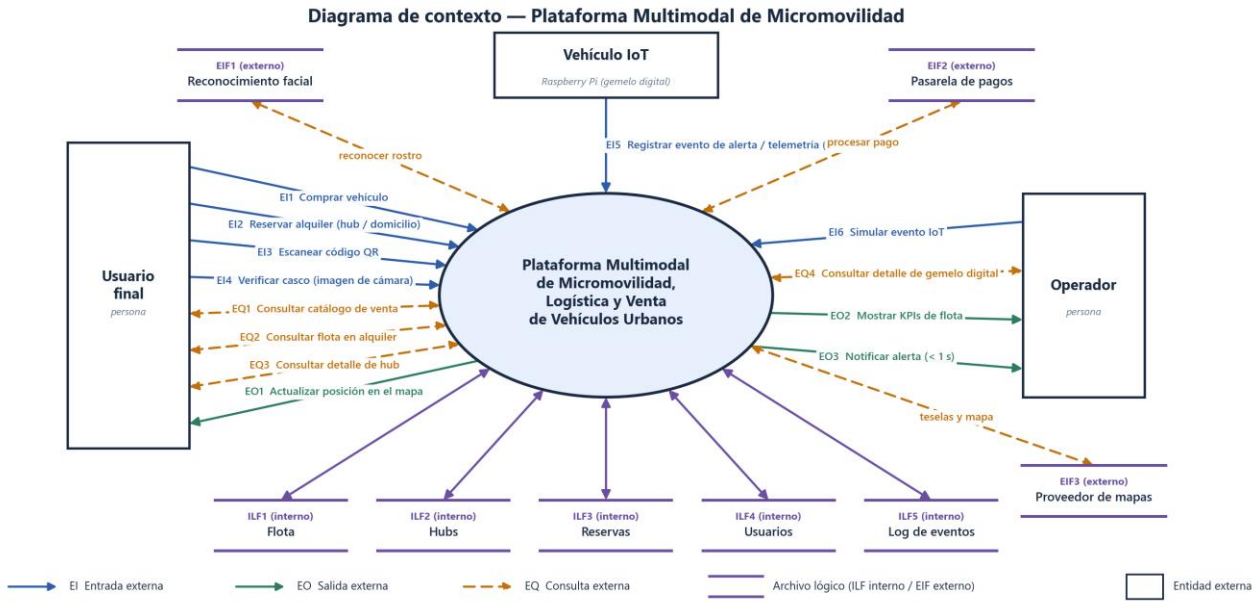


Figura 1. Diagrama de contexto (entradas, salidas, consultas, archivos lógicos e interfaces)

2.2 Funciones del producto

- Landing page comercial con propuesta de valor, servicios y CTA hacia la demo; en dispositivos móviles redirige directamente a la vista de mapa.
- Catálogo y venta de vehículos (patinetas, bicicletas, drones) con precio, condición (nuevo/seminuevo) y especificación técnica.
- Alquiler de vehículos en dos modalidades: recogida en hub (estación a estación) y entrega a domicilio.
- Mapa interactivo multicapa que combina vehículos disponibles para alquiler, hubs con su capacidad/ocupación/carga, y stock disponible para venta.
- Desbloqueo del vehículo mediante escaneo de código QR y verificación del uso del casco por cámara, con reconocimiento facial como mecanismo alterno de autenticación.
- Panel de telemetría (Digital Twin) con KPIs de flota, listado de vehículos conectados y un simulador de eventos IoT para disparar alertas de prueba.

2.3 Tipos de usuario

Usuario final / cliente: persona que compra o alquila un vehículo desde la landing, el catálogo o el mapa de la aplicación.

Operador administrativo: gestiona el catálogo, los hubs y monitorea la flota simulada desde el panel de telemetría (/admin).

Docente evaluador: valida el cumplimiento académico y técnico del avance entregado.

2.4 Restricciones

- El equipo de desarrollo real está compuesto por exactamente tres (3) personas, independientemente del tamaño de equipo sugerido por la estimación teórica inicial.

- En este avance no se dispone de vehículos físicos ni de placas Raspberry Pi conectadas en producción; la flota y su telemetría se simulan como datos en memoria compartidos entre las vistas de mapa y de telemetría.
- La estimación de esfuerzo y costo debe documentarse mediante Puntos de Función y el modelo COCOMO.
- El diseño del sistema debe incluir un Diagrama de Flujo de Datos (DFD) que describa el tránsito de información entre los módulos de la plataforma.
- El prototipo se construye sobre el stack TanStack Start + React 19 + Tailwind CSS 4 ya adoptado por el equipo; cambios de stack quedan fuera del alcance de este avance.

2.5 Supuestos y dependencias

Se asume disponibilidad futura de hardware Raspberry Pi y módulos de cámara para las pruebas de simulación con hardware real, así como acceso a un proveedor de mapas interactivos, a un servicio de reconocimiento facial y a una pasarela de pagos. El cronograma del primer avance depende de la disponibilidad de los tres integrantes del equipo y de la validación oportuna por parte del docente.

2.6 Resumen de riesgos del proyecto

A continuación se listan los riesgos de desarrollo de software de mayor criticidad. El análisis completo — categorías, probabilidad, impacto, matriz de priorización y planes de gestión— se encuentra en el documento anexo “Análisis de Riesgos del Desarrollo”.

ID	Categoría	Descripción	Nivel
RD3	Estimación y equipo	Sobrecarga del equipo real de 3 personas frente a la estimación teórica COCOMO/Puntos de Función.	Alto
RD7	Calidad de software	Falsos positivos/negativos en la lógica de reconocimiento facial para el desbloqueo.	Alto
RD8	Calidad de software	Fallos del trigger de seguridad para detectar caídas o ausencia de casco.	Alto
RD11	Seguridad y cumplimiento	Diseño insuficiente del tratamiento de datos biométricos (Ley 1581 de 2012) en el software.	Alto
RD5	Arquitectura	Acoplamiento de la interfaz a los datos mock en memoria, dificultando la sustitución por un backend real.	Alto

Nota: los planes de Supervisión, Reducción y Gestión (SGR) de estos y de los demás riesgos identificados se detallan exclusivamente en el documento anexo de Análisis de Riesgos.

3. Requisitos específicos

Los requisitos funcionales se identifican con el prefijo RF y se agrupan por módulo, en correspondencia con las vistas del prototipo (/, /app, /admin) y con el modelo de datos compartido (tipos Vehicle y Hub). Cada requisito describe una condición verificable sobre el sistema.

3.1 Requisitos funcionales

3.1.1 Landing page (/)

RF-01 — Landing responsiva: En resolución de escritorio el sistema debe presentar la landing comercial completa (propuesta de valor, servicios, sección "Quiénes somos" y CTA hacia la demo). En resolución móvil debe ingresar directamente a la vista de mapa (/app), dejando la información comercial accesible desde un menú lateral.

3.1.2 Catálogo y venta de vehículos

RF-02 — Listado de catálogo de venta: El sistema debe listar los vehículos en modo venta con los atributos: tipo (patineta, bicicleta o dron), modelo, precio en pesos colombianos, condición (nuevo o seminuevo), especificación técnica y hub donde está disponible.

RF-03 — Compra de vehículo: El usuario debe poder seleccionar un vehículo del catálogo de venta y confirmar su compra desde la aplicación.

3.1.3 Alquiler y reservas

RF-04 — Listado de flota en alquiler: El sistema debe listar los vehículos en modo alquiler con tarifa por minuto, nivel de batería, autonomía/velocidad y ubicación (coordenadas GPS o hub asignado).

RF-05 — Modalidades de alquiler: El sistema debe permitir dos modalidades de alquiler: recogida y devolución en un hub (estación a estación), o entrega del vehículo a domicilio para los vehículos marcados como aptos para entrega. El estado de cada reserva debe registrarse como: solicitada, activa, completada o cancelada.

3.1.4 Mapa interactivo multicapa

RF-06 — Capas del mapa: El mapa debe representar simultáneamente tres capas de información: vehículos disponibles para alquiler, hubs (puntos de recolección) con su ubicación, y stock disponible para venta por hub.

RF-07 — Panel de detalle de hub: Al seleccionar un hub en el mapa, el sistema debe mostrar su capacidad total, número de vehículos ocupados, número de vehículos en carga y los vehículos disponibles para alquiler y venta en ese punto.

RF-08 — Actualización de posición: La posición de cada vehículo simulado en el mapa debe actualizarse a partir de la telemetría (coordenadas GPS) emitida por su gemelo digital, en un intervalo no mayor a 5 segundos desde su generación.

3.1.5 Desbloqueo inteligente

RF-09 — Flujo de desbloqueo en 3 pasos: El desbloqueo del vehículo debe ejecutarse en tres pasos secuenciales: (1) escaneo del código QR del vehículo, (2) verificación del uso del casco mediante cámara, y (3) habilitación de la acción "Iniciar viaje" únicamente cuando la verificación del casco haya sido exitosa.

RF-10 — Reconocimiento facial como mecanismo alterno: El sistema debe ofrecer el reconocimiento facial del usuario registrado como mecanismo alterno de autenticación para el desbloqueo, adicional al escaneo de código QR.

3.1.6 Monitoreo de seguridad IoT durante el viaje

RF-11 — Estados de alerta IoT: Cada vehículo debe reportar en todo momento uno de los siguientes estados de alerta: sin novedad, sin casco, caída/impacto, pérdida de señal GPS o batería crítica.

RF-12 — Generación y registro de alertas: Cuando el estado de un vehículo cambie a un valor distinto de "sin novedad", el sistema debe: (a) generar una notificación inmediata visible para el operador con el tipo de evento, el identificador del vehículo y sus coordenadas GPS; y (b) registrar el evento en un log auditable con marca de tiempo, visible en el panel de telemetría.

3.1.7 Panel de telemetría / Digital Twin (/admin)

RF-13 — Indicadores agregados de flota: El panel debe mostrar como mínimo los siguientes indicadores: número de vehículos conectados, número de hubs activos, batería promedio de la flota y número de alertas activas.

RF-14 — Detalle de gemelo digital: El panel debe permitir seleccionar cualquier vehículo de la flota y consultar el detalle de su gemelo digital: modelo, especificación, nivel de batería, coordenadas GPS, hub asignado, modo (alquiler/venta) y estado de alerta actual.

RF-15 — Simulador de eventos IoT: El panel debe incluir un simulador que permita al operador disparar manualmente cualquiera de los cuatro tipos de alerta (sin casco, caída, pérdida de GPS, batería crítica) sobre un vehículo de la flota en alquiler, con fines de prueba y demostración.

3.1.8 Simulación mediante gemelos digitales (Digital Twins / IoT)

RF-16 — Modelo de gemelo digital: Cada gemelo digital debe representar, como mínimo, los siguientes atributos del vehículo que simula: identificador, tipo, modo, nivel de batería, coordenadas GPS, hub asignado (cuando aplique) y estado de alerta, de forma consistente con el modelo de datos consumido por el mapa y por el panel de telemetría.

RF-17 — Fuente de datos compartida: El mapa interactivo y el panel de telemetría deben leer el estado de la flota y de los hubs desde una única fuente de datos, de manera que un cambio de estado (por ejemplo, una alerta disparada desde el panel) sea consistente entre ambas vistas.

3.1.9 Estimación y gestión del proyecto

RF-18 — Estimación documentada: El proyecto debe documentar su proceso de estimación utilizando Puntos de Función y el modelo COCOMO, presentando una estimación teórica inicial en número de personas y tiempo, y su aterrizaje a la conformación real del equipo de tres (3) personas, en el documento Plan de Desarrollo de Software.

3.1.10 Diagrama de Flujo de Datos (DFD)

RF-19 — DFD del sistema: El diseño del sistema debe incluir un Diagrama de Flujo de Datos que documente el tránsito de información entre los usuarios, el portal de negocio, el módulo de tracking y el módulo de seguridad IoT.

3.2 Requisitos no funcionales

3.2.1 Rendimiento

- RNF-01: la posición de cada vehículo simulado debe reflejarse en el mapa interactivo en un tiempo no mayor a 5 segundos desde su generación en el gemelo digital.
- RNF-02: un evento de alerta disparado desde el simulador de eventos IoT debe reflejarse en la interfaz del panel de telemetría en menos de 1 segundo.

3.2.2 Seguridad

- RNF-03: las plantillas biométricas utilizadas para el reconocimiento facial deben almacenarse cifradas y nunca en texto plano.
- RNF-04: el sistema debe registrar un log auditable de cada evento generado por el trigger de seguridad, con identificador de vehículo, tipo de evento y marca de tiempo.
- RNF-05: las credenciales y tokens de sesión de los usuarios registrados deben almacenarse cifrados, nunca en texto plano.

3.2.3 Disponibilidad

- RNF-06: el portal de negocio y la landing page deben estar disponibles al menos el 95% del tiempo durante el periodo de pruebas del primer avance.

3.2.4 Usabilidad

- RNF-07: el proceso de compra, alquiler y desbloqueo debe poder completarse en un máximo de 3 pasos desde la aplicación.

3.2.5 Escalabilidad

- RNF-08: agregar un nuevo hub o un nuevo vehículo simulado no debe requerir cambios estructurales en el modelo de datos (Vehicle/Hub) ni en los componentes del mapa o del panel de telemetría.

3.2.6 Mantenibilidad

- RNF-09: los módulos de venta, alquiler, tracking y seguridad deben poder actualizarse de manera independiente.
- RNF-10: la fuente de datos simulada de la flota debe poder sustituirse por un backend real sin requerir cambios en los componentes de presentación (mapa, panel de telemetría, flujo de desbloqueo).

3.3 Componentes de terceros

Para este primer avance se contempla el uso de los siguientes componentes de terceros: un proveedor de mapas interactivos para la visualización de la flota en tiempo real, un servicio o librería de reconocimiento facial para el desbloqueo inteligente, un broker MQTT para la comunicación con las placas Raspberry Pi que simularán los gemelos digitales, y una pasarela de pagos para las transacciones de compra y alquiler. El prototipo actual utiliza además las siguientes librerías de código abierto: TanStack Start y TanStack Router/Query (framework y enrutamiento), Tailwind CSS 4 y shadcn/ui (interfaz), Recharts (gráficas) y Framer Motion (animaciones). El uso de cada componente está sujeto a las licencias y restricciones definidas por su respectivo proveedor.

3.4 Interfaces del sistema

3.4.1 Interfaces de usuario

El sistema expone tres interfaces: la landing page pública (/), la aplicación de mapa para el usuario final (/app), desde la cual se realiza la compra, el alquiler, el seguimiento en el mapa y el desbloqueo, y el panel de telemetría para el operador administrativo (/admin).

3.4.2 Interfaces de hardware

El sistema debe interactuar con placas Raspberry Pi que actúan como gemelos digitales de los vehículos, así como con los módulos de cámara y sensores asociados utilizados para la verificación del uso del casco y el monitoreo del viaje.

3.4.3 Interfaces de software

El sistema debe integrarse con el servicio de reconocimiento facial, la pasarela de pagos y el proveedor de mapas interactivos mediante interfaces de programación (API) documentadas.

3.4.4 Interfaces de comunicación

La comunicación entre las placas Raspberry Pi (gemelos digitales) y el backend debe realizarse mediante el protocolo MQTT. La comunicación entre las aplicaciones cliente y el backend debe realizarse mediante servicios REST y, para la actualización de posición en tiempo real sobre el mapa, mediante WebSocket.

3.5 Requisitos de licenciamiento

El software desarrollado, así como las librerías de código abierto utilizadas, deben respetar los términos de licenciamiento de cada componente. El uso del servicio de reconocimiento facial, del proveedor de mapas y de la pasarela de pagos está sujeto a las licencias comerciales o de uso definidas por cada proveedor externo.

3.6 Avisos legales y de derechos de autor

El tratamiento de los datos personales y biométricos (imágenes utilizadas para el reconocimiento facial) recolectados por la plataforma debe cumplir con la Ley 1581 de 2012 y sus decretos reglamentarios. Los derechos de autor sobre el software, la documentación y los artefactos de diseño generados en este proyecto pertenecen a los autores, en el contexto académico de la asignatura Ingeniería de Software II de la Universidad Autónoma de Bucaramanga. Al encontrarse en fase de simulación mediante gemelos digitales, el sistema no debe presentarse como un producto certificado para operación con vehículos físicos reales hasta completar las validaciones de seguridad correspondientes.